

SOLID Design Principles

Part 1

Hierarchy of Pattern Knowledge

Design Pattern

Strategy pattern
defines a family of algorithms,
encapsulates each one, and makes
them interchangeable. Strategy lets
the algorithm vary independently
from clients using it

OO Principles

Encapsulate what varies
Favor composition over inheritance
Program to interface, not implementations

OO Basics

Abstraction
Encapsulation
Polymorphism
Inheritance

Design Smells

Design smells = various signs and symptoms of bad design

Name	Symptoms
Rigidity(경직성)	The system is hard to change, because every time you change one thing, you have to change something else in a never ending succession of changes.
Fragility(취약성)	A change to one part of the system causes it to break in many other, completely unrelated parts.
Immobility(부동성)	It is hard to disentangle the system into components that can be reused in other systems.
Viscosity(점착성)	If it is easier to add a hack than it is to add code that fits into the design, then the system has high viscosity.
Needless Complexity (불필요한 복잡성)	There are lots of very clever code structures that aren't actually necessary right now, but could be very useful one day.
Needless Repetition (불필요한 반복)	The code looks like it was written by two programmers named Cut and Paste.
Opacity(불투명성)	Elucidation of the originator's intent presents certain difficulties related to convolution of expression

Dependency Management

- Design smells are resulted from **mismanaged dependencies**
- Mismanaged dependencies
→ **Tangled mass of couplings (spaghetti code)**
- OO languages provide tools helping managing dependencies
 - Interfaces: break or invert the direction of certain dependencies
 - Polymorphism: allows modules to invoke methods dynamically
 - Lots of power to shape the dependencies, indeed
- **So, how do we want them shaped?**

Object-Oriented Design Principles

- SOLID principles by R.C. Martin
- GRASP patterns/principles by C. Larman
- Program to interfaces, not implementations
- Favor object composition over class inheritance
- Encapsulate what varies
- ...

R.C. Martin's Software design principles (SOLID)

- The **S**ingle-Responsibility Principle (SRP)
- The **O**pen-Closed Principle (OCP)
- The **L**iskov Substitution Principle (LSP)
- The **I**nterface Segregation Principle (ISP)
- The **D**ependency Inversion Principle (DIP)



SINGLE RESPONSIBILITY PRINCIPLE

Just Because You Can, Doesn't Mean You Should

Single Responsibility Principle (SRP)

A class should have one, and only one, reason to change

The Single Responsibility Principle

- Responsibility

- “a contract or obligation of a class”

- **reason to change**

- More responsibilities == More likelihood of change
 - The more a class changes, the more likely we will introduce bugs
 - Changes can impact the others

- SRP: Separate coupled responsibilities into separate classes

- Related measure – Cohesion: how strongly-related and focused are the various responsibilities of a module

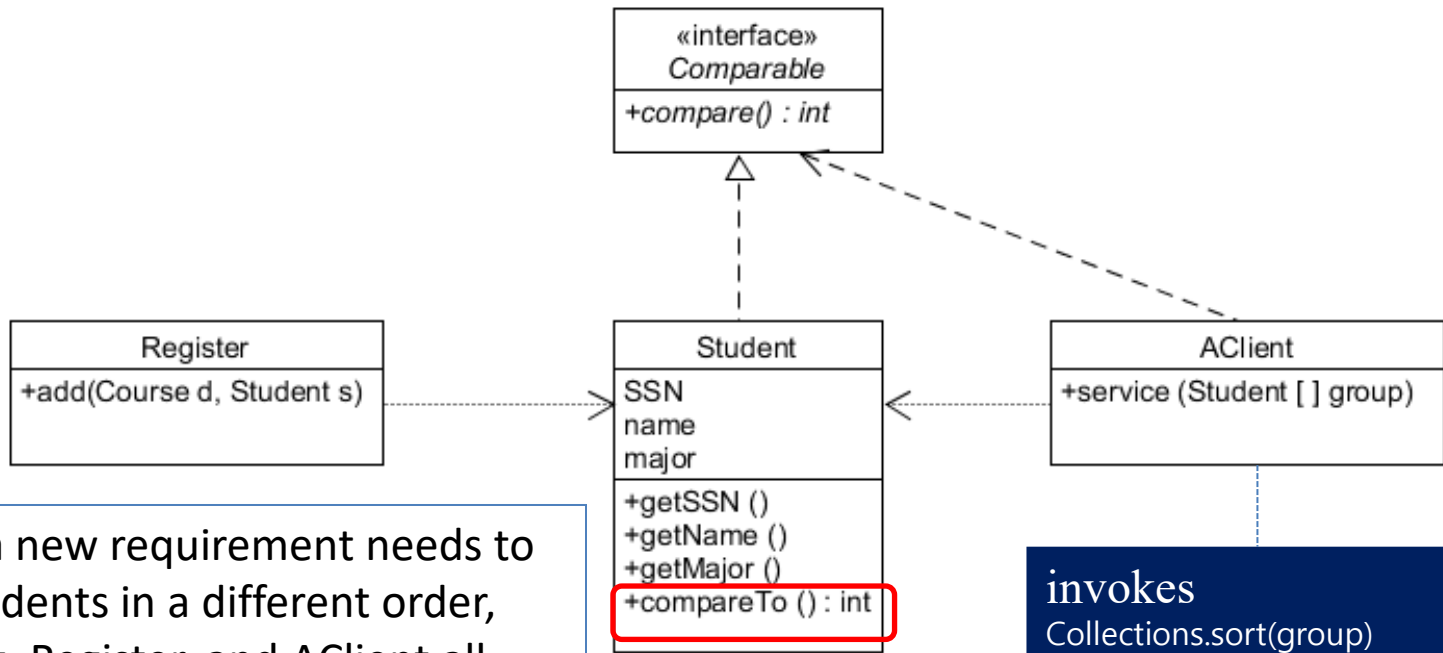
Example of SRP violation: Case 1

- Often we need to sort students by their name, or SSN. So one may make Class Student implement the Java Comparable interface.

```
class Student implements Comparable {  
    ...  
    int compareTo(Object o) { ... }  
    ...  
}
```

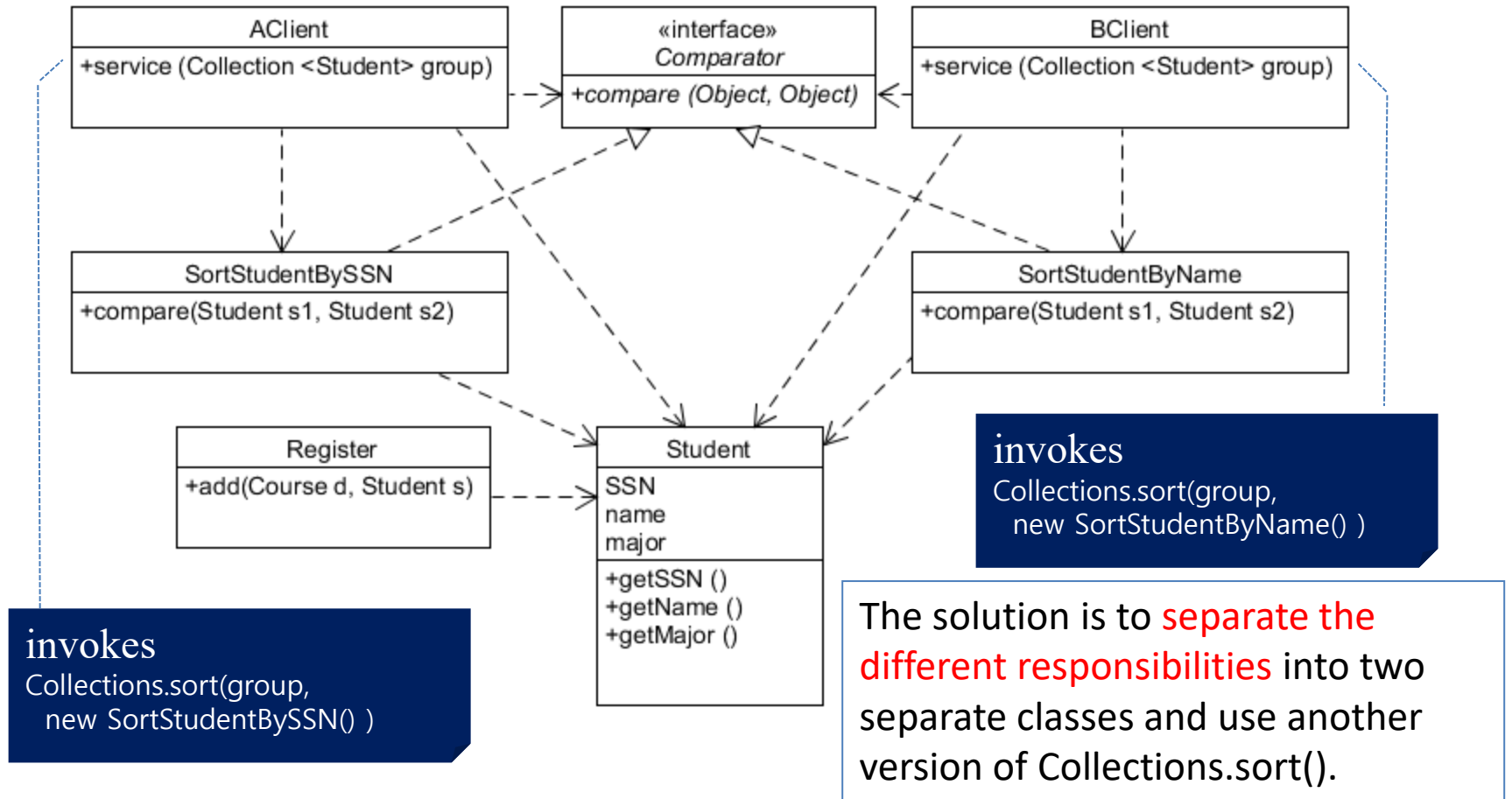
- Student is a business entity, it does not know in what order it should be sorted since the order of sorting is imposed by the client of Student.
- Worse: every time students need to be ordered differently, we have to recompile Student and all its client.
- Cause of the problems: we bundled two separate responsibilities (i.e., student as a business entity with ordering) into one class – **a violation of SRP**

Example of SRP violation

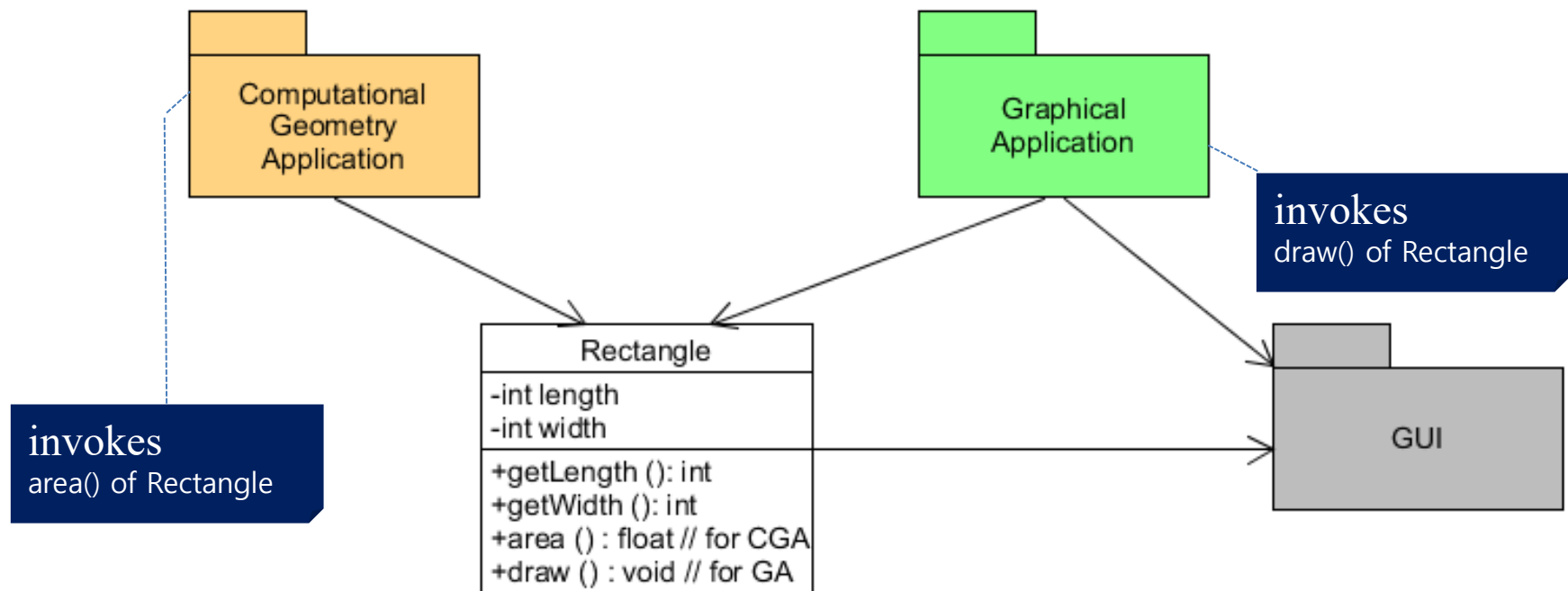


When a new requirement needs to sort students in a different order, `Student`, `Register`, and `AClient` all need to be recompiled, even though `Register` has nothing to do with any ordering of `Students`.

Example of design following SRP

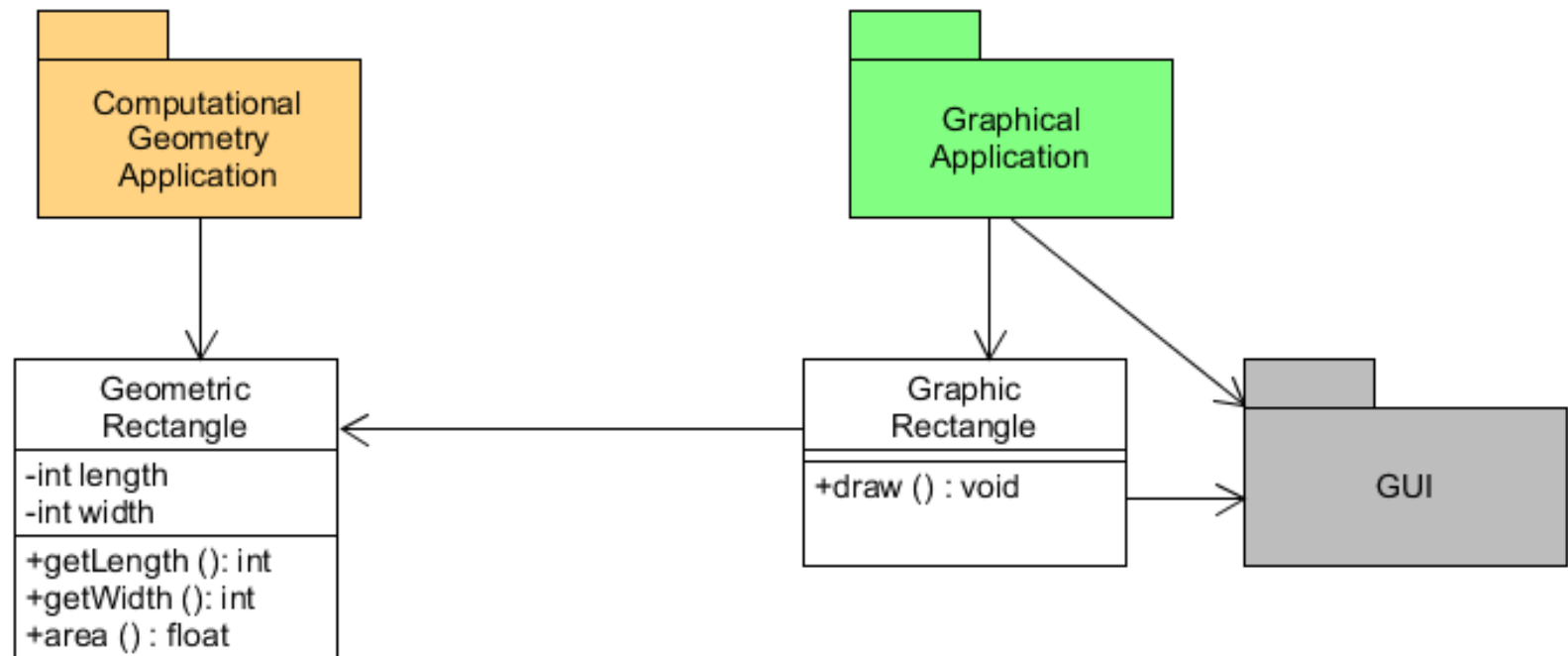


Example of SRP violation: Case 2



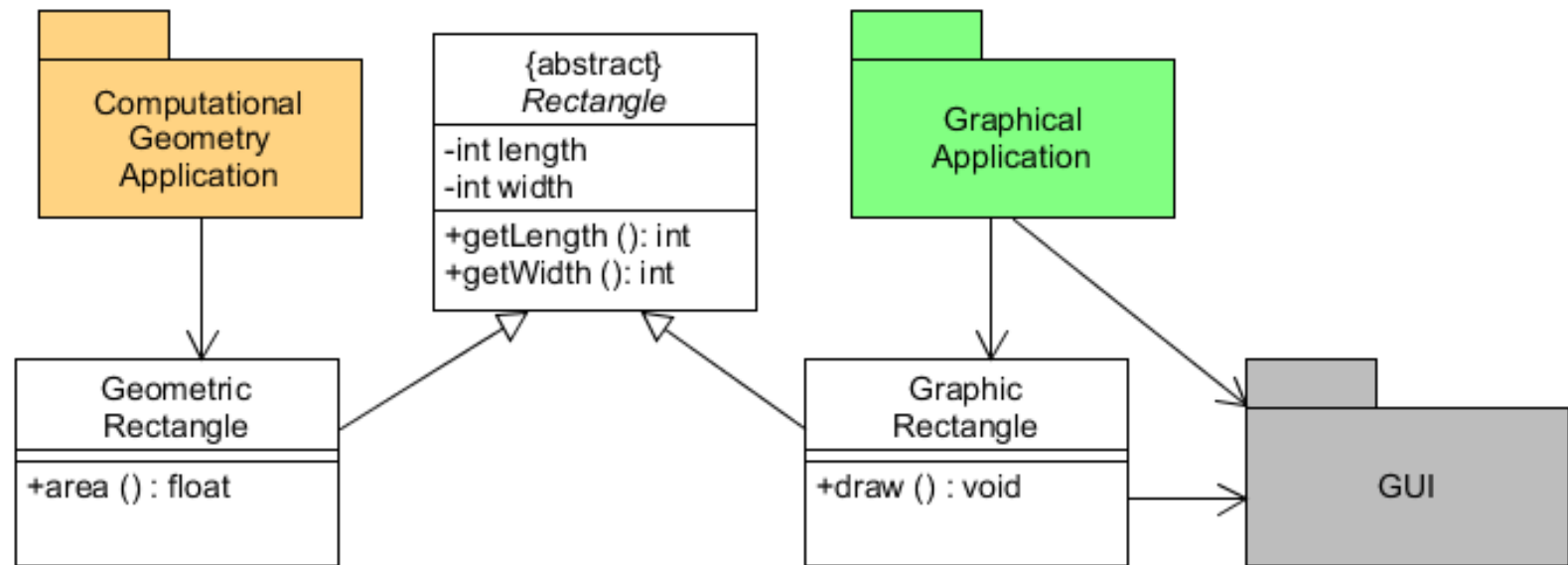
Class **Rectangle** may be forced to make changes from two different unrelated sources. One is from the Computational Geometry Application (CGA). E.g., modifying `area()`. The other is from Graphical Application (GA). E.g., modifying `draw()` for different platforms. A change from either of the two source would still cause the other application to recompile.

Example of design following SRP (not enough yet)



- Package **CGA** is no longer dependent on graphical side of **Rectangle** and thus it becomes independent of package **GUI**. Any change caused by graphical application no longer requires CGA to be recompiled.
- However, any changes from the **CGA** side may cause **GA** to be recompiled.

Example of design following SRP



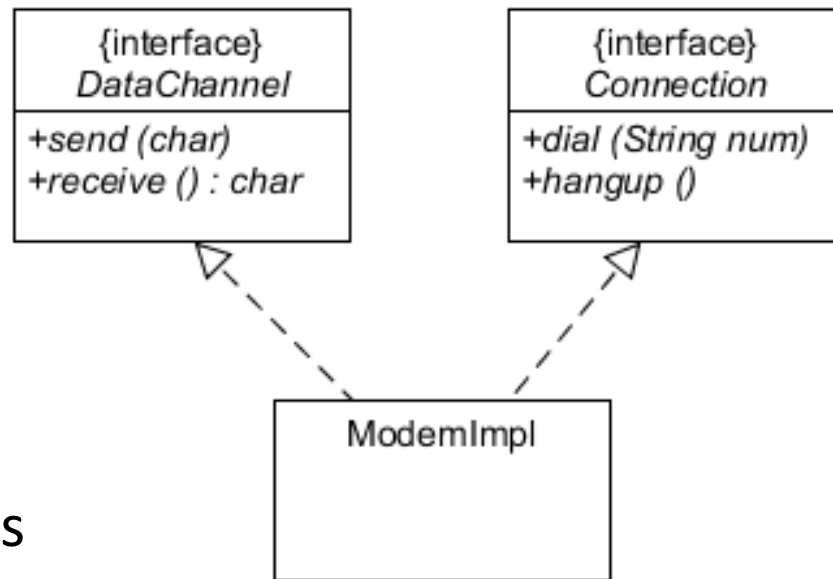
Class **Rectangle** contains the most primitive attributes and operations of rectangles. Classes **GeometricRectangle** and **GraphicRectangle** are independent of each other. A change from either side of **CGA** or **GA**, it would not cause the other side to be recompiled.

Identifying Responsibilities Can be Tricky

- Responsibility (in SRP)
 - A reason for change
 - Note: sometimes hard to see multiple responsibilities

```
interface Modem
{
    public void dial (String num);
    public void hangup ();
    public void send (char c);
    public char receive ();
}
```

Identifying Responsibilities Can be Tricky



- Two responsibilities
 - Connection management
 - Data communication
- Lesson: It depends on how the application is changing!
- Beware: Avoid Needless Complexity
 - If there is no symptom, it is not wise to apply the SRP or any other principle!



OPEN CLOSED PRINCIPLE

Open Chest Surgery Is Not Needed When Putting On A Coat

Open Closed principle (OCP)

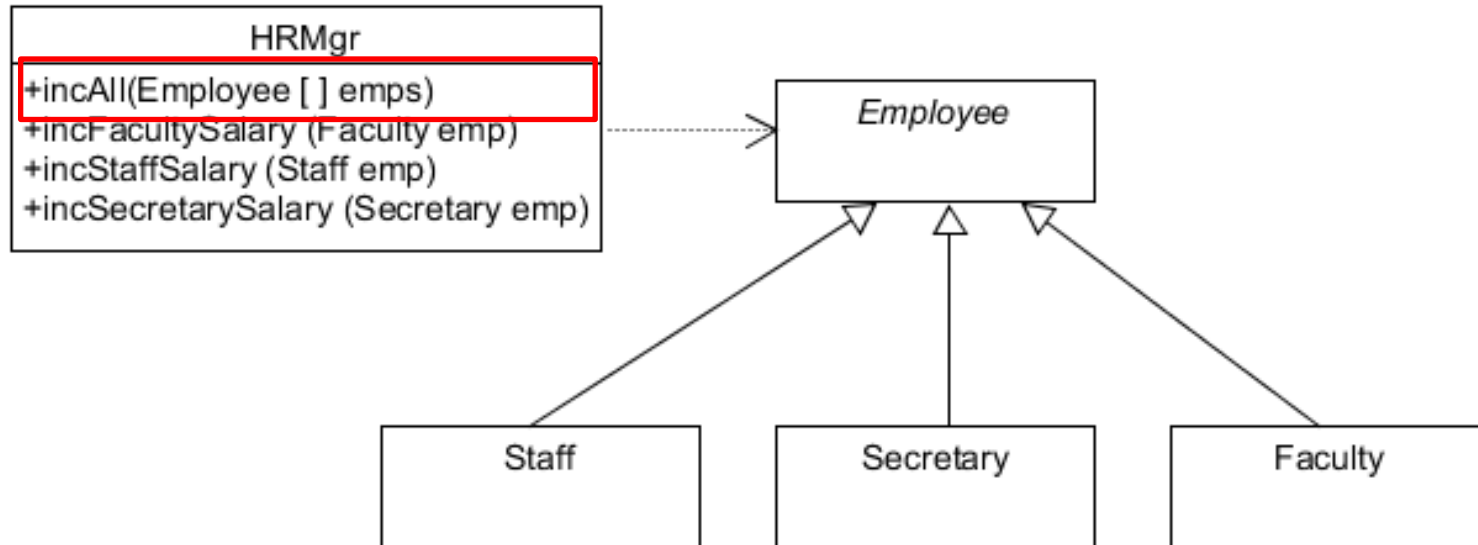
Software entities (classes, modules, functions, etc.) should be open for extension but closed for modification.

You should be able to extend a class's behavior, without modifying it.

Conforming to OCP

- Open for extension
 - Behavior of the module can be extended
 - We are able to change what the module does
- Closed for modification
 - Extending behavior does not result in excessive modification such as architectural changes of the module
- Recall the PV in GRASP
- Violation Indicator: Design Smell of Rigidity
 - A single change to a program results in a **cascade of changes to dependent modules**

Bad Design Example



```
void incAll(Employee[] emps) {
    for (int i = 0; i < emps.size(); i++) {
        if (emps[i].empType == FACULTY)
            incFacultySalary((Faculty)emps[i]);
        else if (emps[i].empType == STAFF)
            incStaffSalary((Staff)emps[i]);
        else if (emps[i].empType == SECRETARY)
            incSecretarySalary((Secretary)emps[i]);
    }
}
```

Problems of Bad Design

- Rigid
 - Adding new employee type requires significant changes
- Fragile
 - Many switch/case or if/else statements
 - Hard to find and understand

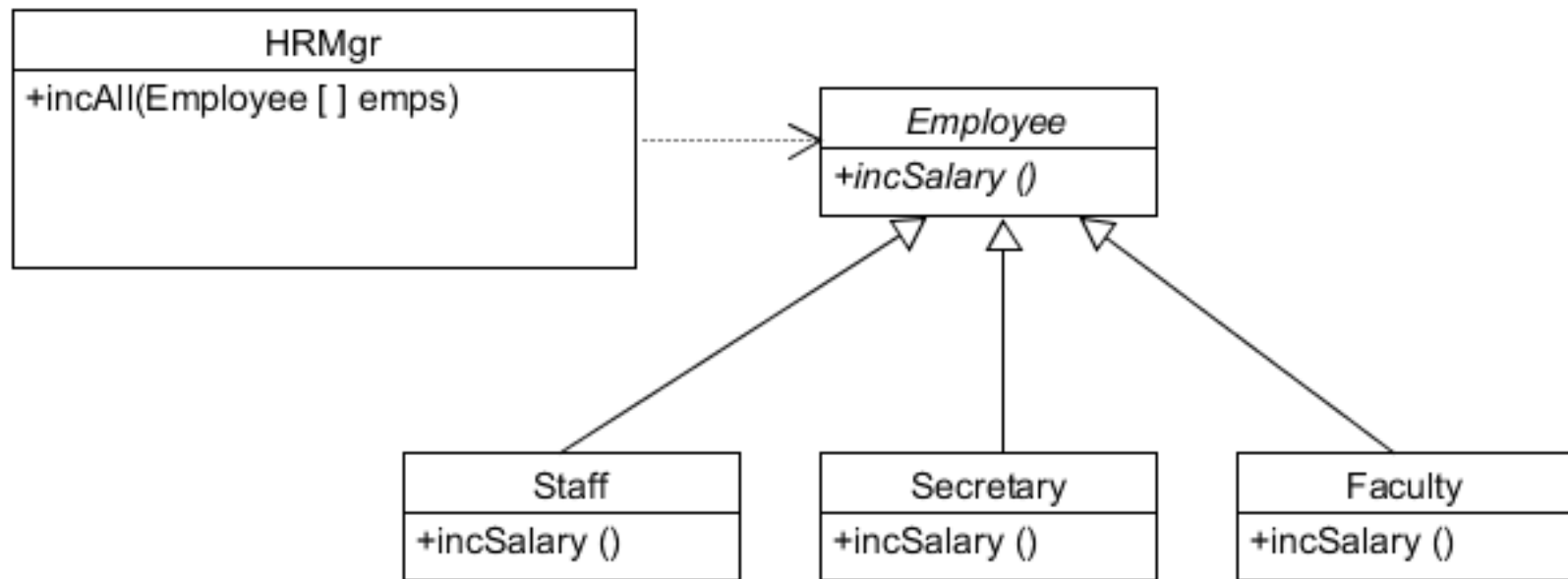
```
void incAll(Employee[] emps) {  
    for (int i = 0; i < emps.size(); i++) {  
        if (emps[i].empType == FACULTY)  
            incFacultySalary((Faculty)emps[i]);  
        else if (emps[i].empType == STAFF)  
            incStaffSalary((Staff)emps[i]);  
        else if (emps[i].empType == SECRETARY)  
            incSecretarySalary((Secretary)emps[i]);  
        else if (emps[i].empType == ENGINEER)  
            incEngineersSalary((Engineer)emps[i]);  
    }  
}  
void incEngineersSalary (Engineer e) { ... }
```

Problems of Bad Design

- Rigid
 - Adding new employee type requires significant changes
- Fragile
 - Many switch/case or if/else statements
 - Hard to find and understand
- Immobile
 - To reuse incAll() → we need Faculty, Staff, Secretary, too!
 - What if we need just Faculty and Staff only?

```
void incAll(Employee[] emps) {  
    for (int i = 0; i < emps.size(); i++) {  
        if (emps[i].empType == FACULTY)  
            incFacultySalary((Faculty)emps[i]);  
        else if (emps[i].empType == STAFF)  
            incStaffSalary((Staff)emps[i]);  
        else if (emps[i].empType == SECRETARY)  
            incSecretarySalary((Secretary)emps[i]);  
    }  
}
```

Better Design



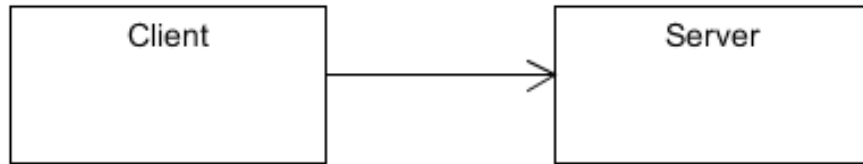
```
void incAll(Employee[] emps) {
    for (int i = 0; i < emps.size(); i++)
        emps[i].incSalary();
}
```

- When Engineer is added, `incAll()` does not even need to recompile.
- This design is open to extension, closed for modification.

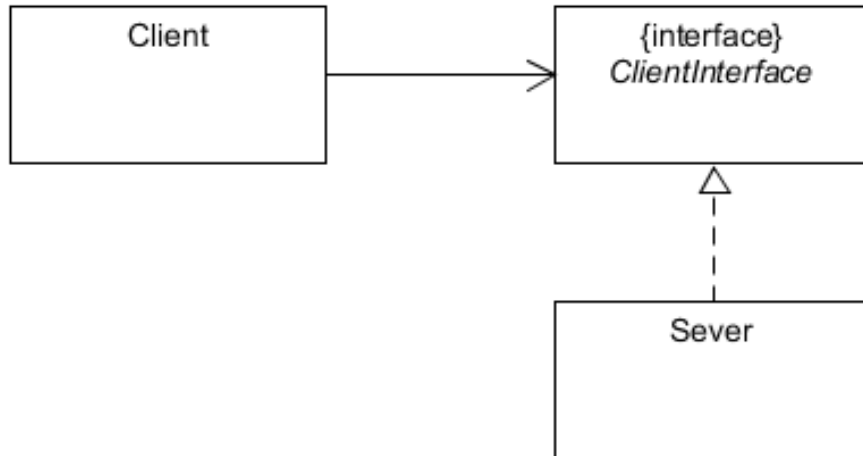
Abstraction and Polymorphism for OCP

- Abstractions
 - Fixed and yet represent an unbounded group of possible behaviors
 - Abstract base class : fixed
 - All the possible derived classes : unbounded group of possible behaviors
- (Dynamic) Polymorphism
 - Method overriding
 - Dynamic binding
 - Program the class to interfaces (or abstract classes) not to implementation (concrete classes)

Abstraction and Polymorphism for OCP



What if we need to use other servers ?



Abstraction + Polymorphism
Client depends on only Interface

Anticipating Future Changes

- Strategy is needed
 - Choose the kinds of changes against which to close design
 - Guess the most likely kinds of changes, and then construct abstractions to protect him from those changes.
- Beware: Consider the cost!
 - Conforming to OCP is expensive -> run time cost가 비쌘
 - Time and effort to create appropriate abstractions -> 이해하기 힘들
 - Abstractions also increase complexity

-> OCP가 필요할때만 사용, 무분별하게 사용시 오히려 비효율적

Anticipating Future Changes

- Do not put hooks in for changes that might happen
 - "Fool me once, shame on you. Fool me twice, shame on me."
 - Initially write the code expecting it to not change.
 - When a change occurs, implement the abstractions that protect from future changes of that kind.
 - It's better to take the first hit as early as possible.
 - We want to know what kind of changes are likely before going too far in the development.
 - Use TDD and listen to the tests.